Listing 34.7 Program.cs for a worker service that saves exchange rates using EF Core

```
using Microsoft.EntityFrameworkCore;

IHost host = Host.CreateDefaultBuilder(args)                          ❶
    .ConfigureServices((hostContext, services) =>                     ❷
    {
        services.AddHttpClient<ExchangeRatesClient>();                ❸
        services.AddHostedService<ExchangeRatesHostedService>();      ❸

        var connectionString = hostContext.Configuration              ❹
            .GetConnectionString("SqlLiteConnection"))                ❹

        services.AddDbContext<AppDbContext>(options =>                ❺
          options.UseSqlite(connectionString));                       ❺
    })
    .Build();                                                         ❻

host.Run();                                                           ❼
```

❶ Creates an IHostBuilder using the default helper

❷ Configures your DI services

❸ Adds services to the IServiceCollection

❹ IConfiguration can be accessed from the HostBuilderContext parameter.

❺ Adds services to the IServiceCollection

❻ Builds an IHost instance

❼ Runs the app and waits for shutdown

The changes in Program.cs to use the generic host instead of minimal hosting are the most obvious differences between a worker service and an ASP.NET Core app, but there are some important differences in the .csproj project file too. The following listing shows the project file for a worker service that uses `IHttpClientFactory` and EF Core, and highlights some of the differences with a similar ASP.NET Core application.

Listing 34.8 Project file for a worker service